AKTE 3 · ERMITTLUNG 3.2 · DS 6

Die Statistik lügt nicht

LÖSUNG · NUR LEHRKRAFT

STADTFEST

FALL-EINSTIEG *Die Kasse rechnet richtig – und trotzdem fehlen 312,50 €. Am Kaffeemobil stehen Bons mit Positionen der Menge 0. Edda Lorenz: „Daten lügen nicht. Abfragen manchmal schon."*

BEWEISSTÜCK 1 Die Zeitlupe: In dieser Reihenfolge wertet die Datenbank aus

Geschrieben wird `SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY … LIMIT`. Ausgewertet wird anders:



Deshalb kann WHERE noch nichts über Gruppen wissen, und ein Alias aus dem SELECT ist in HAVING noch unbekannt (in ORDER BY dagegen schon). Nach EH steht in HAVING immer die Aggregatfunktion selbst.

BEWEISSTÜCK 2 WHERE filtert Zeilen vorher – HAVING filtert Fächer nachher

Welche Stände hatten mehr als 40 Barzahlungen?

```
SELECT Stand.StandNr, Stand.Name, COUNT(*) AS Barbons
FROM Stand, Bon
WHERE Stand.StandNr = Bon.StandNr
  AND Bon.Zahlart = 'bar'
GROUP BY Stand.StandNr, Stand.Name
HAVING COUNT(*) > 40
ORDER BY Barbons DESC;
```

StandNr	Name	Barbons
4	Kaffeemobil	122
5	Bierwagen	67
1	Grillhütte	54

Schritt	Was passiert mit den Bon-Karten?
WHERE `Bon.Zahlart = 'bar'`	Kartenzahler setzen sich hin, **bevor** Fächer gebildet werden (Zeilenbedingung).
GROUP BY	Die übrigen Bons gehen in ihr Stand-Fach.
HAVING `COUNT(*) > 40`	Fächer mit höchstens 40 Bons werden **danach** aussortiert (Gruppenbedingung).

BEWEISSTÜCK 3 Top-n mit LIMIT – und das Gleichstand-Problem

`ORDER BY … DESC LIMIT n` liefert die ersten n Zeilen nach der Sortierung. Wer hatte die meisten Schichten?

```
SELECT Helfer.HelferNr, Helfer.Vorname, Helfer.Nachname,
       COUNT(*) AS Schichten
FROM Helfer, Schicht
WHERE Helfer.HelferNr = Schicht.HelferNr
GROUP BY Helfer.HelferNr, Helfer.Vorname, Helfer.Nachname
ORDER BY Schichten DESC, Helfer.HelferNr;
```

HelferNr	Vorname	Nachname	Schichten
33	Enno	Kolthoff	4
35	Arne	Thölken	4
40	Mats	Vosgerau	4
51	Ida	Rohlfs	4
1	Pauline	Hedden	3
…	…	…	…

Mit `LIMIT 3` fiele eine der vier Personen mit 4 Schichten weg – welche, entscheidet der Zufall der Sortierung. Eine Top-n-Frage ist nur eindeutig, wenn an Platz n **kein Gleichstand** besteht. Sonst besser fragen: „alle Personen mit mindestens 4 Schichten" (HAVING).

BEWEISSTÜCK 4 Vervielfachung im Verbund

Bon 486 (das Beweisstück) hat **eine** Position: Kaffee, Menge 0. Verbindet man zusätzlich Schicht nur über den Stand, passt jede Schicht des Kaffeemobils ab 21 Uhr zu diesem Bon:

```
SELECT Bon.BonNr, Position.ProduktNr, Schicht.HelferNr,
       Schicht.Beginn
FROM Bon, Position, Schicht
WHERE Bon.BonNr = Position.BonNr
  AND Bon.StandNr = Schicht.StandNr
  AND Bon.BonNr = 486
  AND Schicht.Beginn >= '2026-08-29 21:00'
ORDER BY Schicht.Beginn;
```

BonNr	ProduktNr	HelferNr	Beginn
486	11	7	2026-08-29 21:00
486	11	23	2026-08-29 22:00
486	11	7	2026-08-29 23:00

Aus einer Position werden drei Zeilen – `SUM(Produkt.Preis)` würde den Kaffee dreimal zählen (7,50 € statt 2,50 €). **Regel:** Nur die Tabellen verbinden, die für die Frage nötig sind, und jede Verbundbedingung prüfen. Ein Ergebnis, das „plausibel aussieht", ist noch kein Beweis.

BEWEISSTÜCK 5 Analysieren und Vorgehen beschreiben – Satzbausteine

Analyse im Sachzusammenhang (nicht: „wählt aus, gruppiert, zählt …"):

- „Die Abfrage liefert **für jede/jeden …** (Gruppe mit Schlüssel und Name) **die/den …** (Aggregat mit Bedeutung) …"
- „… **berücksichtigt werden nur** … (WHERE, Zeilen) …"
- „… **ausgegeben werden nur** Gruppen, die/deren … (HAVING) …"
- „… **sortiert nach** … (ORDER BY), **nur die ersten** n (LIMIT)."

Vorgehen in Worten (Abitur: „Beschreiben Sie Ihr Vorgehen …"): ① benötigte **Tabellen** → ② **Verbund** über Primär-/Fremdschlüssel → ③ **Zeilenfilter** (WHERE) → ④ **Gruppierung** nach dem Schlüssel → ⑤ **Gruppenfilter** (HAVING) → ⑥ **Sortierung/Begrenzung** und Ausgabe.

Ermittlungsaufträge

NR.	STUFE	ORT	AUFTRAG

1 ★☆☆ PAPIER

Alle sechs Bedingungen stammen aus Abfragen, die nach Stand gruppieren. Kreuze an, ob sie in WHERE oder in HAVING gehören.

Bedingung	WHERE	HAVING
a) `Helfer.Jahrgang < 1970`	☐	☐
b) `COUNT(*) > 20`	☐	☐
c) `Produkt.Preis >= 4`	☐	☐
d) `MIN(Bon.Zeitpunkt) < '2026-08-28 17:00'`	☐	☐
e) `Helfer.Verein = 'Shantychor Weserstedt'`	☐	☐
f) `SUM(Position.Menge * Produkt.Preis) > 500`	☐	☐

LÖSUNG

WHERE: a), c), e) – sie prüfen eine einzelne Zeile. HAVING: b), d), f) – sie enthalten eine Aggregatfunktion und beziehen sich auf ein ganzes Fach.

Typische Fehler:

◆ **P1** d) in WHERE, weil „es um Zeitpunkte geht" – entscheidend ist MIN(…).

2 ★★☆ PAPIER

Analysiere die Abfrage: Beschreibe im Sachzusammenhang des Stadtfests, welche Information sie liefert. Nutze die Satzbausteine aus Beweisstück 5.

```
SELECT Helfer.Verein, COUNT(*) AS Bons
FROM Helfer, Bon
WHERE Helfer.HelferNr = Bon.HelferNr
  AND Bon.Zeitpunkt LIKE '2026-08-28%'
GROUP BY Helfer.Verein
HAVING COUNT(*) >= 40
ORDER BY Bons DESC;
```

LÖSUNG

Die Abfrage liefert für jeden **Verein** die Anzahl der **Bons, die Mitglieder dieses Vereins am Freitag (28.08.) kassiert haben**. Ausgegeben werden nur Vereine mit **mindestens 40** solchen Bons, absteigend nach Anzahl. Personen ohne Verein bilden eine eigene Gruppe (NULL).

Verein	Bons
Förderverein Schulzentrum	69
NULL	60
Freiwillige Feuerwehr Weserstedt	45

Typische Fehler:

◆ **P10** Syntax nacherzählen („verbindet Helfer und Bon, gruppiert nach Verein …") statt die Information zu beschreiben.

◆ **P5** Die NULL-Gruppe übersehen.

3 ★★☆ LAB

Welche Stände haben einen durchschnittlichen Produktpreis **unter 3 €**? Gib StandNr, Name und den Durchschnittspreis aus. Teste in der Konsole und gib das Ergebnis an.

LÖSUNG

```
SELECT Stand.StandNr, Stand.Name,
       AVG(Produkt.Preis) AS Durchschnittspreis
FROM Stand, Produkt
WHERE Stand.StandNr = Produkt.StandNr
GROUP BY Stand.StandNr, Stand.Name
HAVING AVG(Produkt.Preis) < 3;
```

StandNr	Name	Durchschnittspreis
4	Kaffeemobil	2,75
7	Losbude	2,5

NR.	STUFE	ORT	AUFTRAG

Typische Fehler:
- **P1** `WHERE AVG(Produkt.Preis) < 3` – Aggregat in WHERE.
- **P1** Alias in HAVING (`HAVING Durchschnittspreis < 3`) – läuft in SQLite, ist nach EH aber nicht vorgesehen.

4 ★★☆ PAPIER

Edda fragt: „Welcher Stand hatte am Festsamstag (29.08.) den höchsten Umsatz mit **Kartenzahlungen**?“ **Beschreibe** dein Vorgehen in Worten – Tabellen, Verbund, Filter, Gruppierung, Sortierung. Eine fertige Abfrage ist nicht verlangt.

LÖSUNG

1. Tabellen **Stand, Bon, Position, Produkt** (Umsatz = Menge × Preis).
2. Verbund: Stand.StandNr = Bon.StandNr, Bon.BonNr = Position.BonNr, Position.ProduktNr = Produkt.ProduktNr.
3. Zeilenfilter: Zahlart 'Karte' und Zeitpunkt am 2026-08-29.
4. Gruppieren nach StandNr (und Name), je Stand `SUM(Position.Menge * Produkt.Preis)`.
5. Absteigend nach dieser Summe sortieren, nur die erste Zeile ausgeben – vorher prüfen, ob an Platz 1 ein Gleichstand besteht.

Zur Kontrolle:

```
SELECT Stand.StandNr, Stand.Name,
       SUM(Position.Menge * Produkt.Preis) AS Umsatz
FROM Stand, Bon, Position, Produkt
WHERE Stand.StandNr = Bon.StandNr
  AND Bon.BonNr = Position.BonNr
  AND Position.ProduktNr = Produkt.ProduktNr
  AND Bon.Zahlart = 'Karte'
  AND Bon.Zeitpunkt LIKE '2026-08-29%'
GROUP BY Stand.StandNr, Stand.Name
ORDER BY Umsatz DESC
LIMIT 1;
```

StandNr	Name	Umsatz
4	Kaffeemobil	302

Typische Fehler:
- **P10** Nur „mit SQL die Daten abfragen“ – ohne Tabellen und Bedingungen.
- **P4** Bons zählen statt Umsatz summieren.

5 ★★★ PAPIER

Juri soll die Anzahl der Bons je Stand ermitteln. Seine Abfrage liefert für das Kaffeemobil 316 Bons – Frauke Melchers hat aber nur 177 Bons in der Kasse. **Beurteile** die Abfrage und verändere sie so, dass sie korrekt ist. *(Schwerpunkt eA)*

```
SELECT Stand.StandNr, Stand.Name, COUNT(*) AS Bons
FROM Stand, Bon, Position
WHERE Stand.StandNr = Bon.StandNr
  AND Bon.BonNr = Position.BonNr
GROUP BY Stand.StandNr, Stand.Name
ORDER BY Bons DESC;
```

StandNr	Name	Bons
4	Kaffeemobil	316
5	Bierwagen	150
1	Grillhütte	145
…	…	…

LÖSUNG

Juri verbindet zusätzlich **Position**. Ein Bon mit drei Positionen erscheint dann in drei Zeilen; `COUNT(*)` zählt **Positionen** statt Bons (Vervielfachung). Für die Frage wird Position nicht gebraucht:

NR.	STUFE	ORT	AUFTRAG

```
SELECT Stand.StandNr, Stand.Name,
       COUNT(*) AS Bons
FROM Stand, Bon
WHERE Stand.StandNr = Bon.StandNr
GROUP BY Stand.StandNr, Stand.Name
ORDER BY Bons DESC;
```

StandNr	Name	Bons
4	Kaffeemobil	177
5	Bierwagen	103
1	Grillhütte	90
6	Crêpes-Stand	51
7	Losbude	48
3	Saftbar	44
2	Fischbrötchen-Kutter	41

Typische Fehler:

◆ **P2** Unnötige Tabelle im Verbund vervielfacht Zeilen – das Ergebnis sieht plausibel aus, ist aber zu hoch.

ABI-ECKE Im Stil des Abiturs (Operator analysieren, AFB II)

Gegeben ist die folgende SQL-Abfrage zur Datenbank Stadtfest. **Analysieren** Sie die Abfrage, indem Sie beschreiben, welche Informationen sie im Sachzusammenhang liefert. [4 BE]

```
SELECT Produkt.Bezeichnung, SUM(Position.Menge) AS Stueck
FROM Bon, Produkt, Position
WHERE Bon.BonNr = Position.BonNr
  AND Position.ProduktNr = Produkt.ProduktNr
  AND Bon.Zeitpunkt LIKE '2026-08-29%'
GROUP BY Produkt.ProduktNr, Produkt.Bezeichnung
HAVING SUM(Position.Menge) > 50
ORDER BY Stueck DESC;
```

LÖSUNG

Die Abfrage liefert für jedes **Produkt**, das am **Samstag, 29.08.**, verkauft wurde, die Bezeichnung und die **verkaufte Stückzahl** dieses Tages. Ausgegeben werden nur Produkte mit **mehr als 50** verkauften Stück, das meistverkaufte zuerst.

Bezeichnung	Stueck
Los	100
Currywurst	56
Kaffee	51

Je 1 BE: Bezug je Produkt und Stückzahl (SUM der Menge, nicht Anzahl der Bons) · Einschränkung auf den Samstag · Gruppenbedingung mehr als 50 Stück · Sortierung absteigend.

Typische Fehler:

◆ **P10** „Die Abfrage verbindet drei Tabellen und gruppiert …" – Syntax statt Sachzusammenhang.

◆ **P4** „Anzahl der Verkäufe" statt verkaufter Stück.

SPUR IM LAB Ermittlung 3.2 „Die Statistik lügt nicht"

Aufgaben: A3-E2-01 … A3-E2-08 · Cold Case A3-E2-CC · danach Verhör 3 (Schulkiosk) · **Werkzeug:** Zeitlupe (aus jeder Diagnose)

A3-E2-06 und A3-E2-08 sind Analyse-Aufgaben: Deine Antwort bleibt auf deinem Gerät, du hakst die Kernpunkte selbst ab.

COLD CASE · OHNE PUNKTE

Die Kasse stimmt doch

Zeige mit zwei Abfragen, dass die Umsätze je Stand und der Gesamtumsatz zusammenpassen. Erläutere dann, warum der Fehlbetrag in keiner der beiden Zahlen auftaucht – und mit welcher Abfrage man ihn sichtbar macht.

LÖSUNG

```
SELECT Stand.StandNr, Stand.Name,
       SUM(Position.Menge * Produkt.Preis) AS Umsatz
FROM Stand, Bon, Position, Produkt
WHERE Stand.StandNr = Bon.StandNr
  AND Bon.BonNr = Position.BonNr
  AND Position.ProduktNr = Produkt.ProduktNr
GROUP BY Stand.StandNr, Stand.Name;
```

StandNr	Name	Umsatz
1	Grillhütte	1080
2	Fischbrötchen-Kutter	632,5
3	Saftbar	452,5
4	Kaffeemobil	1041
5	Bierwagen	876
6	Crêpes-Stand	534
7	Losbude	407

```
SELECT SUM(Position.Menge * Produkt.Preis) AS Gesamtumsatz
FROM Position, Produkt
WHERE Position.ProduktNr = Produkt.ProduktNr;
```

Gesamtumsatz
5023

Die Summe der Standumsätze ergibt genau 5023 €. Positionen mit Menge 0 tragen 0 × Preis = 0 € zum Umsatz bei – die Kasse „rechnet richtig". Sichtbar wird der Fehlbetrag erst über den **Warenwert** der Positionen mit Menge 0 (Listenpreis statt Menge × Preis):

```
SELECT Stand.Name, COUNT(*) AS Nullpositionen,
       SUM(Produkt.Preis) AS Warenwert
FROM Stand, Produkt, Position
WHERE Stand.StandNr = Produkt.StandNr
  AND Produkt.ProduktNr = Position.ProduktNr
  AND Position.Menge = 0
GROUP BY Stand.StandNr, Stand.Name;
```

Name	Nullpositionen	Warenwert
Grillhütte	1	4,5
Saftbar	1	2,5
Kaffeemobil	110	312,5
Bierwagen	1	3,5